

# Ecosistema Hadoop y Spark

|                                                  |    |
|--------------------------------------------------|----|
| Introducción.....                                | 3  |
| ¿Qué es Hadoop?.....                             | 5  |
| Un poco de historia.....                         | 5  |
| Componentes principales de Hadoop .....          | 6  |
| ¿Qué es Apache Spark?.....                       | 7  |
| Ventajas de Spark sobre Hadoop MapReduce .....   | 7  |
| RDDs y DataFrames.....                           | 9  |
| ¿Cuándo usar cada uno?.....                      | 9  |
| Usando DataFrames sus métodos.....               | 10 |
| 1. Carga de datos .....                          | 10 |
| 2. Inspección de datos.....                      | 11 |
| 3. Selección de columnas .....                   | 12 |
| 4. Filtrado de filas.....                        | 12 |
| 5. Creación de columnas nuevas.....              | 13 |
| 6. Agrupamiento y agregación.....                | 14 |
| 7. Ordenación.....                               | 15 |
| 8. Eliminación de duplicados .....               | 15 |
| 9. Unión de DataFrames .....                     | 15 |
| 10. Guardado de resultados.....                  | 15 |
| Instalación del entorno de trabajo PySpark ..... | 16 |
| Código ejemplo PySpark (Dataframes) .....        | 17 |
| BLOQUE 1: Instalación .....                      | 18 |
| BLOQUE 2 .....                                   | 18 |
| BLOQUE 3 .....                                   | 19 |
| BLOQUE 4 .....                                   | 19 |

## Introducción

---

En la unidad anterior hablamos del concepto “Big Data”, sus características (5V) y su arquitectura general:

Captura de datos → Almacenamiento → Procesamiento → Analisis → Visualización.

Además de entender el problema y la necesidad de nuevas tecnologías para tratar grandes datos.

Pero nos quedamos en lo conceptual!!, sabemos que necesitamos procesar enormes volúmenes de datos, pero no explicamos cómo hacerlo. Ahora vamos a conocer dos de las tecnologías más importantes que hacen posible ese **procesamiento**: **Hadoop** y **Spark**.

Y veremos cómo **Spark** nos ofrece **herramientas de programación** como:

- **RDDs** (Resilient Distributed Dataset)
- **DataFrames**

que nosotros, como desarrolladores, podemos usar directamente.

Resumiendo... ahora pasamos de lo **teórico (conceptos y arquitectura)** a lo **tecnológico (frameworks y APIs reales)**.

Por lo tanto, ya somos capaces de responder a las preguntas:

- “¿Qué tienen en común Netflix, Spotify y Amazon?”

Respuesta: **Big Data** – procesan millones de datos en tiempo real. Datos que crecen más rápido que nuestras bases de datos tradicionales.

- Y “¿YouTube?”

Respuesta: **Big Data** – procesan millones de datos en tiempo real. Datos que crecen más rápido que nuestras bases de datos tradicionales. P.e: 500 horas de vídeo subidas a YouTube cada minuto. Millones de tuits diarios.

Y también conocemos el **problema: una sola máquina no puede con tanto volumen**.

Por este motivo en esta unidad vamos a hablar de las tecnologías más importantes que hacen posible el procesamiento de este gran volumen de datos; **Hadoop** y **Spark**.



## ¿Qué es Hadoop?

**Hadoop** es un **framework de código abierto** que permite el procesamiento distribuido de grandes conjuntos de datos a través de un **cluster\*** de ordenadores, utilizando modelos de programación simples.

Componentes principales:

- **HDFS** (sistema de archivos distribuido).
- **MapReduce** (motor de procesamiento).
- **YARN** (gestor de recursos).



Características:

- Su desarrollo es coordinado por la compañía Apache Foundation.
- Su motor clásico es MapReduce, que permite procesar datos de forma distribuida, pero como veremos es lento y complejo de programar.

### Un poco de historia

**Hadoop** vio la luz en el año 2004 cuando un ingeniero de la compañía Google realizó un documento con técnicas que permitían gestionar grandes cantidades de datos de manera que los dividía cada vez en problemas más pequeños, para que a su vez fueran más fácilmente asumibles con el fin de encontrar la raíz del problema y en consecuencia la solución. Se terminó de desarrollar por completo en el año 2008.

En resumen podríamos decir que Hadoop nació como una iniciativa de código abierto (Open source) que trataba de resolver los problemas asociados al Big Data y el Data Science, actualmente es la plataforma open source líder en Big Data. Tanto es así que en el argot de los datos masivos Hadoop casi se ha convertido en sinónimo de Big Data.

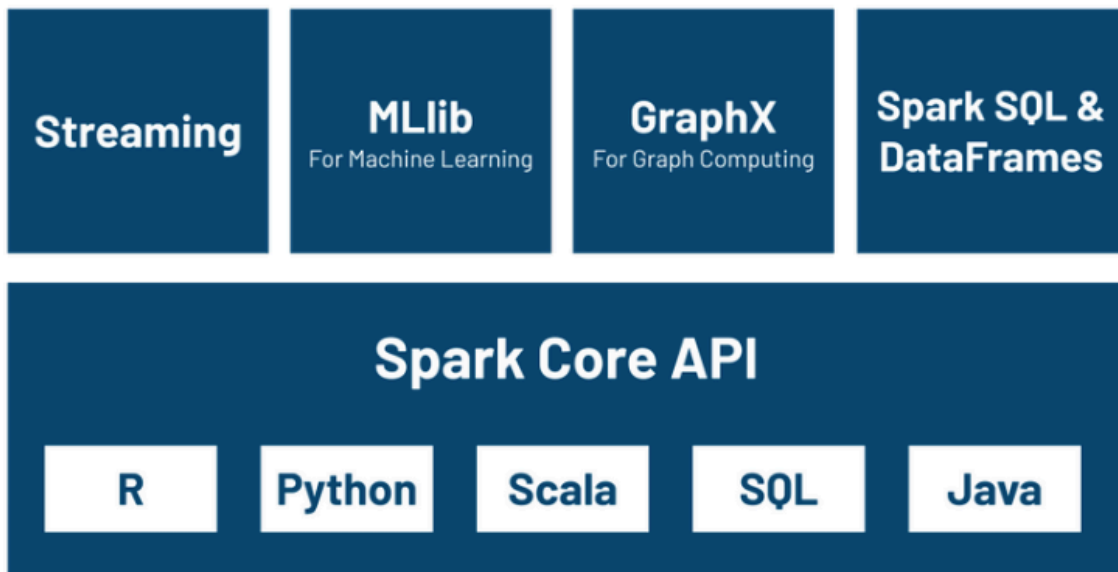
\***cluster**: Un clúster de ordenadores es un conjunto de ordenadores (nodos) conectados en red que trabajan juntos como si fueran un solo sistema.

## Componentes principales de Hadoop

- **HDFS:** Hadoop Distributed File System, sistema de archivos distribuido. Es el sistema de almacenamiento de Hadoop, diseñado para guardar grandes volúmenes de datos repartidos en un clúster de ordenadores.
- **MapReduce:** Modelo de programación para el procesamiento paralelo de datos. Creado por Google y adoptado por Hadoop para procesar grandes volúmenes de datos en un clúster de ordenadores. Su objetivo es **dividir un problema grande en muchos problemas pequeños**, procesarlos en paralelo y luego combinar los resultados.
- **YARN:** Yet Another Resource Negotiator, gestiona los recursos del clúster. Es el gestor de recursos de Hadoop, encargado de controlar y coordinar el uso de CPU, memoria y almacenamiento en todo el clúster.

## ¿Qué es Apache Spark?

**Apache Spark** es un motor de procesamiento de datos en memoria que permite ejecutar programas de forma más rápida que **Hadoop MapReduce**. **Soporta procesamiento por lotes y en tiempo real.**



*Componentes de Spark*

### Ventajas de Spark sobre Hadoop MapReduce

- **Mayor velocidad** (procesamiento en memoria).
- **APIs** en múltiples lenguajes (Python, Scala, Java, R).
- **Librerías** integradas para SQL (Spark SQL), Machine Learning (MLlib), procesamiento de grafos (GraphX) y datos en streaming.

**Apache Spark** ofrece varias ventajas clave sobre **Hadoop MapReduce**, lo que ha llevado a muchas organizaciones a adoptarlo como su principal motor de procesamiento de datos.

A continuación se muestra la tabla de comparación:

| Característica                         | Spark                                                       | Hadoop MapReduce                              |
|----------------------------------------|-------------------------------------------------------------|-----------------------------------------------|
| <b>Velocidad</b>                       | Mucho más rápido (hasta 100x en memoria).                   | Más lento debido al acceso constante a disco. |
| <b>Procesamiento en memoria</b>        | Usa memoria RAM para evitar I/O innecesario.                | Accede al disco después de cada operación.    |
| <b>Modelo de programación</b>          | API más sencilla y expresiva (RDD, DataFrame).              | Verboso, basado en pares <clave, valor>.      |
| <b>Procesamiento en tiempo real</b>    | Soporta streaming con Spark Streaming/Structured Streaming. | Solo procesamiento batch (por lotes).         |
| <b>Manejo de tareas complejas</b>      | Permite pipelines con múltiples transformaciones.           | Difícil de encadenar tareas complejas.        |
| <b>Librerías integradas</b>            | MLlib (ML), GraphX (grafos), Spark SQL, etc.                | No tiene librerías avanzadas integradas.      |
| <b>Tolerancia a fallos</b>             | Usa DAGs y registros de transformación.                     | Basado en el sistema de archivos HDFS.        |
| <b>Reutilización de datos en cache</b> | Se puede almacenar RDDs en memoria para reusarlos.          | No permite caching entre tareas.              |

Resumiendo:

1. **Spark** es ideal cuando se necesita:
  - Procesamiento rápido.
  - Trabajo interactivo o iterativo (como en Machine Learning).
  - Análisis en tiempo real.
  - Un ecosistema unificado (ML, SQL, streaming, grafos).
2. **Hadoop MapReduce** aún puede ser útil si:
  - Solo se necesita **procesamiento batch**.
  - Los recursos son limitados (Spark necesita más RAM).
  - Se quiere un sistema más sencillo o ya existente.



## RDDs y DataFrames

---

**Spark** nos ofrece **herramientas de programación** como:

- **RDDs** (Resilient Distributed Dataset)
- **DataFrames**

Ambos son abstracciones para manejar datos distribuidos, pero tienen diferencias importantes en rendimiento, facilidad de uso y optimización.

- **RDD (Resilient Distributed Dataset)**: estructuras de datos más básicas de Spark que permiten manejar datos distribuidos y tolerantes a fallos.
- **DataFrames**: RDD + esquema (tablas). Es una abstracción de alto nivel encima de RDD. Es decir, es una estructura de datos tabular similar a una tabla SQL, optimizada para rendimiento.

### ¿Cuándo usar cada uno?

Usa **RDDs** cuando:

- Necesitas bajo nivel de control (por ejemplo, manipulación compleja de datos).
- Estás procesando **datos no estructurados** o de **formatos muy personalizados**.

Usa **DataFrames** cuando:

- Puedes trabajar con **datos estructurados**.
- Quieres mejor rendimiento y menos código.
- Estás haciendo análisis, limpieza, joins o ML (ML se refiere a Machine Learning (aprendizaje automático), y Spark incluye una poderosa librería para esto llamada MLlib).

## Usando DataFrames sus métodos

### 1. Carga de datos

- `read.csv()` → leer CSV
- `read.parquet()` → leer Parquet
- `read.json()` → leer JSON

Ejemplo:

```
# Importar SparkSession desde PySpark
from pyspark.sql import SparkSession

# Crear la sesión
spark = SparkSession.builder.appName("Spark Example").getOrCreate()

# Leer CSV
df = spark.read.csv("archivo.csv", header=True, inferSchema=True)
```

# Unidad 2

## 2. Inspección de datos

- `show()` → muestra las filas

| nombre | edad | ciudad    |
|--------|------|-----------|
| Ana    | 25   | Madrid    |
| Luis   | 35   | Barcelona |
| Pedro  | 40   | Madrid    |
| Laura  | 28   | Valencia  |

- `printSchema()` → imprime el esquema (tipos de columnas)

```
root
|-- nombre: string (nullable = true)
|-- edad: integer (nullable = true)
|-- ciudad: string (nullable = true)
```

- `columns` → lista de nombres de columnas

```
['nombre', 'edad', 'ciudad']
```

- `describe()` → estadísticas básicas

| summary | nombre | edad              | ciudad    |
|---------|--------|-------------------|-----------|
| count   | 4      | 4                 | 4         |
| mean    | NULL   | 32.0              | NULL      |
| stddev  | NULL   | 6.782329983125268 | NULL      |
| min     | Ana    | 25                | Barcelona |
| max     | Pedro  | 40                | Valencia  |

Ejemplo:

```
df.show()
df.printSchema()
print(df.columns)
df.describe().show()
```

### 3. Selección de columnas

- `select()` → elegir columnas
- `drop()` → quitar columnas

Ejemplo:

```
df.select("columna1", "columna2").show()
df.drop("columna3").show()
```

### 4. Filtrado de filas

- `filter()` o `where()` → aplicar condiciones

Ejemplo:

```
df.filter(df["edad"] > 30).show()

# Otra manera de referenciar a la columna edad es utilizando col, así:
df.filter(col("edad") > 30).show() # necesario from pyspark.sql.functions import col

df.where("edad > 30").show()
```

## 5. Creación de columnas nuevas

- withColumn() → agregar o reemplazar columnas

Ejemplo – Agregar columna nueva ingreso calculada con otras dos columnas, quantity y price:

```
from pyspark.sql.functions import col  
df = df.withColumn("ingreso", col("quantity") * col("price"))
```

Ejemplo – Agregar columna nueva pais con valor fijo España:

```
from pyspark.sql.functions import lit  
df = df.withColumn("pais", lit("España"))
```

## 6. Agrupamiento y agregación

- `groupBy()` → agrupar filas
- `agg()` → aplicar funciones agregadas
- Funciones: `sum()`, `avg()`, `count()`, `max()`, `min()`

Ejemplo 1:

```
from pyspark.sql.functions import sum, avg
df.groupBy("product").agg(sum("quantity"), avg("price")).show()
```

Si `df` fuera:

| product | quantity | price |
|---------|----------|-------|
| A       | 2        | 10    |
| A       | 3        | 20    |
| B       | 1        | 5     |

El resultado sería:

| product | sum(quantity) | avg(price) |
|---------|---------------|------------|
| A       | 5             | 15.0       |
| B       | 1             | 5.0        |

Ejemplo 2:

```
from pyspark.sql.functions import sum, avg, count
```

```
df.groupBy("product") \
    .agg(
        sum("quantity").alias("total_quantity"),
        avg("price").alias("avg_price"),
        count("*").alias("num_rows")
    ) \
    .show()
```

## 7. Ordenación

- `orderBy()`

Ejemplo – Tres formas de hacer el order by:

```
df.orderBy("price").show() #1.  
df.orderBy(df["price"].desc()).show() #2.  
  
from pyspark.sql.functions import  
df.orderBy(col("price").desc()).show() #3. necesario el import para col
```

## 8. Eliminación de duplicados

- `dropDuplicates()`

Ejemplo:

```
df.dropDuplicates().show()
```

## 9. Unión de DataFrames

- `join()`

Ejemplo:

```
df1= spark.read.csv("archivo.csv", header=True, inferSchema=True)  
df2 = spark.read.csv("archivo2.csv", header=True, inferSchema=True)  
  
df1.join(df2, on="id", how="inner")
```

## 10. Guardado de resultados

- `write.csv()`, `write.parquet()`

Ejemplo:

```
df.write.csv("salida.csv", header=True)
```

## Instalación del entorno de trabajo PySpark

---

### Notas importantes para trabajar con PySpark en Visual Code:

- Asegúrate de tener instalado Python
- Asegúrate de tener PySpark instalado: `pip install pyspark`
- Si vas a leer un CSV, asegúrate de que la ruta "datos.csv" sea relativa al archivo .py que estás ejecutando, o usa una ruta absoluta.
- Para pruebas rápidas, crear DataFrames desde listas o diccionarios es más sencillo que depender de archivos externos.

Ver documento (ANEXO I – Instalación del entorno de trabajo PySpark) para instalación de pySpark en distintos IDEs (Visual Code y NetBeans).



## Código ejemplo PySpark (Dataframes)

PySpark es el API de Python para Apache Spark. Permite escribir código Spark usando Python. (Nota: instalar python y pyspark desde consola con -> pip install pyspark). Ejemplo básico de cómo usar PySpark para leer y procesar datos:

```
# Importar SparkSession desde PySpark
from pyspark.sql import SparkSession

# Crear una sesión de Spark
spark = SparkSession.builder.appName("Ejemplo").getOrCreate()

# Leer un archivo CSV como DataFrame con cabecera e inferencia de tipos
df = spark.read.csv("datos.csv", header=True, inferSchema=True)

# Filtrar las filas donde la edad es mayor a 30,
# luego agrupar por ciudad y contar cuántos registros hay por ciudad
df.filter(df["edad"] > 30).groupBy("ciudad").count().show()
```

El archivo **datos.csv** debe tener un formato como este:

```
nombre,edad,ciudad
Ana,25,Madrid
Luis,35,Barcelona
Pedro,40,Madrid
Laura,28,Valencia
```

Si el archivo contiene los datos anteriores, la salida sería algo como:

```
| ciudad |count|
+-----+-----+
| Madrid | 1    |
| Barcelona | 1    |
```

## BLOQUE 1: Instalación

---

Instala PySpark en tu entorno y carga un archivo CSV de ejemplo. Realiza operaciones de filtrado, agrupación y ordenación utilizando DataFrames.

## BLOQUE 2

---

Carga el archivo CSV de ventas de ejemplo (ventas.csv) y después de leerlo, calcula las ventas totales por producto.

El archivo **ventas.csv** tiene un formato como este:

```
date,product,quantity,price
2024-01-01,A,10,5.0
2024-01-01,B,20,7.5
2024-01-02,A,15,5.0
```

La salida esperada es esta:

```
+-----+-----+
|product|total_ventas|
+-----+-----+
|  A  |      125.0|
|  B  |      150.0|
+-----+-----+
```

NOTA: Filtrar ventas con quantity > 0 y agrupa por producto sumando ingresos (quantity\*price)

## BLOQUE 3

---

Carga el archivo CSV de ventas de ejemplo (ventas.csv) y después de leerlo, calcula el total de unidades vendidas por día.

El archivo **ventas.csv** tiene un formato como este:

```
date,product,quantity,price
2024-01-01,A,10,5.0
2024-01-01,B,20,7.5
2024-01-02,A,15,5.0
```

La salida esperada es esta:

```
+-----+-----+
| date      | total_unidades |
+-----+-----+
| 2024-01-01 | 30              |
| 2024-01-02 | 15              |
+-----+-----+
```

NOTA: Agrupa por date. Aplica sum sobre quantity. Renombra la suma como “total\_unidades”.

## BLOQUE 4

---

1. ¿Qué diferencias hay entre Hadoop y Spark? Explica con tus palabras.
2. Enumera y describe brevemente los tres componentes principales de Hadoop.
3. ¿Por qué Spark es más rápido que MapReduce?
4. Explica qué es un RDD y en qué se diferencia de un DataFrame.